

LINQ Inner Join in C# with Examples

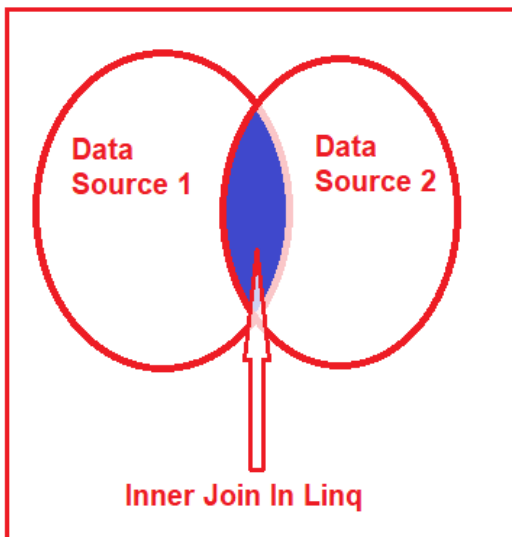
In this article, I will discuss the **LINQ Inner Join in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the basics of [LINQ Join in C#](#). This is the most common join used in real-time applications. At the end of this article, you will understand the following pointers:

1. [What is LINQ Inner Join in C#?](#)
2. [How to Implement Inner Join in LINQ?](#)
3. [Examples to Understand LINQ Inner Join in C#](#)
4. [Using LINQ Join Method to Perform Inner Join](#)
5. [Using LINQ Query Syntax to Perform Inner Join](#)
6. [Optimizing Performance for Large Datasets using LINQ Inner Join](#)
7. [When Should We Use LINQ Inner Join in C#?](#)

What is LINQ Inner Join in C#?

As per the Microsoft documentation, **An Inner Join produces a result set in which each element of the first collection appears once for every matching element in the second collection. If an element in the first collection does not have any matching element in the second collection, then it does not appear in the result set.**

In simple words, we can say that the LINQ Inner Join returns only the matching elements from both data sources while the non-matching elements are removed from the result set. So, if you have two data sources, and when you perform the Inner Join, only the matching elements in both data sources are included in the result set. For a better understanding of Inner Join, please look at the following image, which shows the pictorial representation of Inner Join.



Note: A common property should exist in both data sources while performing the Inner Join, and based on the common property, the matching records are retrieved.

How to Implement Inner Join in LINQ?

We need to use the LINQ Join Method to implement an Inner Join. The Join Method operates on two data sources or sequences, or you can also say two collections, such as an inner collection and an outer collection. The Join Method returns a new collection containing data from both collections, the same as the SQL join, based on a common property. Two overloaded versions of the Join method are available in LINQ, and their signature is as follows.

```
public static IEnumerable<TResult> Join<TOuter, TInner, TKey, TResult>(this IEnumerable<TOuter> outer, IEnumerable<TInner> inner,
Func<TOuter, TKey> outerKeySelector, Func<TInner, TKey> innerKeySelector, Func<TOuter, TInner, TResult> resultSelector);

public static IEnumerable<TResult> Join<TOuter, TInner, TKey, TResult>(this IEnumerable<TOuter> outer, IEnumerable<TInner> inner,
Func<TOuter, TKey> outerKeySelector, Func<TInner, TKey> innerKeySelector, Func<TOuter, TInner, TResult> resultSelector,
IEqualityComparer<TKey> comparer);
```

The only difference between the above two overloaded versions is that the second version takes a comparer as an extra parameter. So, while working with LINQ Join Method (with LINQ Method Syntax) or join operator (with LINQ Query Syntax), we must understand the following five things.

1. **Outer Data Source:** This is the first data source or collection involved in the Join Operation.
2. **Inner Data Source:** This is the second data source or collection involved in the Join Operation.
3. **Outer Key Selector:** This will be the common key in the outer data source.
4. **Inner Key Selector:** This will be the common key in the inner data source.
5. **Result Selector:** Project the data into a result set, including the properties from both Inner and Outer Data Sources.

Examples to Understand LINQ Inner Join in C#:

Let us understand how to Implement LINQ Inner Join in C#. For this, we will use the following two model classes: Employee and Address. So, create a class file named **Employee.cs** and copy and paste the following code. This class has three properties: ID, Name, and AddressId. We have also created one method to return a collection of Employees, which will be our first data source for the Inner Join.

```
using System.Collections.Generic;
namespace LINQJoin
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int AddressId { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            return new List<Employee>()
            {
                new Employee { ID = 1, Name = "Preety", AddressId = 1 },
                new Employee { ID = 2, Name = "Priyanka", AddressId = 2 },
                new Employee { ID = 3, Name = "Anurag", AddressId = 3 },
                new Employee { ID = 4, Name = "Pranaya", AddressId = 4 },
                new Employee { ID = 5, Name = "Hina", AddressId = 5 },
                new Employee { ID = 6, Name = "Sambit", AddressId = 6 },
                new Employee { ID = 7, Name = "Happy", AddressId = 7 },
            }
        }
    }
}
```

```

        new Employee { ID = 8, Name = "Tarun", AddressId = 8 },
        new Employee { ID = 9, Name = "Santosh", AddressId = 9 },
        new Employee { ID = 10, Name = "Raja", AddressId = 10},
        new Employee { ID = 11, Name = "Sudhanshu", AddressId = 11}
    };
}
}
}

```

Now, create another class file named **Address.cs** and copy and paste the following code. This class has 2 properties, i.e., ID and AddressLine. We have also created one method to return a collection of addresses, which will be our second data source for the Inner Join.

```

using System.Collections.Generic;
namespace LINQJoin
{
    public class Address
    {
        public int ID { get; set; }
        public string AddressLine { get; set; }
        public static List<Address> GetAllAddresses()
        {
            return new List<Address>()
            {
                new Address { ID = 1, AddressLine = "AddressLine1"},
                new Address { ID = 2, AddressLine = "AddressLine2"},
                new Address { ID = 3, AddressLine = "AddressLine3"},
                new Address { ID = 4, AddressLine = "AddressLine4"},
                new Address { ID = 5, AddressLine = "AddressLine5"},
                new Address { ID = 9, AddressLine = "AddressLine9"},
                new Address { ID = 10, AddressLine = "AddressLine10"},
                new Address { ID = 11, AddressLine = "AddressLine11"},
            };
        }
    }
}

```

Note: In Real-Time Applications, you will fetch data from a database. Here, we will not focus on how to fetch the data from a database. Rather, we will focus on how to perform the inner join. So, we created the required data sources (i.e., a list of employees and addresses) with the hard-coded data. In this article, I will show how to join two data sources, and in our next article, I will explain how to join multiple data sources using the LINQ Join method.

In both data sources, the common property is the Address ID, i.e., the AddressId property of the Employee data source, and the ID property of the Address data source is the common property. As you can see, we have 11 records in the employee data source and 8 in the addresses data source. Further, if you notice, some of the data are present in both the data sources.

Using LINQ Join Method in C#:

Let us see how to use the LINQ Join Method in C#. We want to fetch the employee's name and corresponding address in an anonymous type. But here, we need to fetch the employees who have addresses. If one employee does not have the corresponding address, we don't want that employee in our result set. So, basically, we need to perform an Inner Join between the Employee and Address Data Sources. In this case, you can take any data source as the Outer data source and any data source as the Inner Data Source.

Using LINQ Method Syntax to Perform Inner Join in C#:

The following code uses the Method Syntax to perform LINQ Inner Join in C#. Here, you can see we are using the Join Method. **Employee.GetAllEmployees()** is our Outer Data Source and **Address.GetAllAddresses()** is our Inner Data Source. Here, we are accessing the employee's information using the employee variable and the addresses using the address variable. Further, you can notice we are specifying the outer key selector as **AddressId** using Lambda Expression and specifying the Inner Key Selector as **ID** using the lambda expression. These Outer and Inner Key Selectors are nothing but the common properties in both data sources. Finally, we are projecting the result to an anonymous type and fetching the employee name to the EmployeeName property and AddressLine to the AddressLine property.

```
Employee.GetAllEmployees() //Outer Data Source
.Join(
Address.GetAllAddresses(), //Inner Data Source
employee => employee.AddressId, // Outer Key Selector
address => address.ID, // Inner Key selector
(employee, address) => new //Projecting the data into a result set
{
    EmployeeName = employee.Name,
    AddressLine = address.AddressLine
}).ToList();
```

The complete example code is given below. Once the Join Operation is done, you can access the elements using a for each loop, which is shown in the example below. The following code is self-explained, so please go through the comment lines.

```
using System.Linq;
using System;
namespace LINQJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Inner Between Employees and Addresses Data Sources
            var JoinUsingMS = Employee.GetAllEmployees() //Outer Data Source
                .Join( //Performing LINQ Inner Join
                    Address.GetAllAddresses(), //Outer Data Source
                    employee => employee.AddressId, //Outer Key Selector
                    address => address.ID, //Inner Key selector
```

```

        (employee, address) => new //Projecting the data in
Anonymous Type
    {
        EmployeeName = employee.Name,
        AddressLine = address.AddressLine
    }).ToList();
//Accessing the Result using For Each Loop
foreach (var employee in JoinUsingMS)
{
    Console.WriteLine($"Name :{employee.EmployeeName}, Address :
{employee.AddressLine}");
}
Console.ReadLine();
}
}
}

```

Output:

```

Name :Preety, Address : AddressLine1
Name :Priyanka, Address : AddressLine2
Name :Anurag, Address : AddressLine3
Name :Pranaya, Address : AddressLine4
Name :Hina, Address : AddressLine5
Name :Santosh, Address : AddressLine9
Name :Raja, Address : AddressLine10
Name :Sudhanshu, Address : AddressLine11

```

As you can see in the above output, it only fetches the matching records from both data sources. Instead of projecting the result to an anonymous type, can we project the result to a named type? Yes, it is possible. Let us see how we can do this. First, create a class file named **EmployeeAddress.cs** with the required properties you want in the result set. As per our requirement, we have created the class with two properties.

```

namespace LINQJoin
{
    class EmployeeAddress
    {
        public string EmployeeName { get; set; }
        public string AddressLine { get; set; }
    }
}

```

Next, modify the Main method of the Program class as follows. Here, you can see we are projecting the result to the above-created EmployeeAddress type. Further, we have changed the Inner and Outer Data Source order to ensure we get the expected output when performing the Inner Join Operations in LINQ. Here, we are making the Addresses as the Outer Data Source and Employees as the Inner Data Source.

```

using System.Linq;
using System;

```

```

namespace LINQJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Inner Between Employees and Addresses Data Sources
            var JoinUsingMS = Address.GetAllAddresses() //Outer Data Source
                .Join( //Performing LINQ Inner Join
                    Employee.GetAllEmployees(), //Outer Data Source
                    address => address.ID, //Outer Key Selector
                    employee => employee.AddressId, //Inner Key selector
                    (address, employee) => new EmployeeAddress //Project
data to named type i.e. EmployeeAddress
                {
                    EmployeeName = employee.Name,
                    AddressLine = address.AddressLine
                }).ToList();
            //Accessing the Result using For Each Loop
            foreach (var employee in JoinUsingMS)
            {
                Console.WriteLine($"Name :{employee.EmployeeName}, Address :
{employee.AddressLine}");
            }
            Console.ReadLine();
        }
    }
}

```

With the above changes in place, run the application, and you will also get the same output as the previous example, as shown in the below image.

```

Name :Preety, Address : AddressLine1
Name :Priyanka, Address : AddressLine2
Name :Anurag, Address : AddressLine3
Name :Pranaya, Address : AddressLine4
Name :Hina, Address : AddressLine5
Name :Santosh, Address : AddressLine9
Name :Raja, Address : AddressLine10
Name :Sudhanshu, Address : AddressLine11

```

Let's rewrite the same example using LINQ Query Syntax.

Using LINQ Query Syntax to Perform Inner Join in C#:

LINQ allows the **join** operator to perform the join using Query syntax in C#. Performing the join operation using query syntax is similar to performing the join in SQL. I personally preferred to use Query Syntax instead of Method Syntax as it is syntactically similar to SQL Join.

To better understand how to perform the Join using LINQ Query Syntax, please look at the following code snippet. First, we need to fetch the data from the Outer Data Source, and then we need to use the “**join**” operator followed by the second data source. In the below code, we are accessing the first data source using variable **emp** and the second data source using the variable **address**. Like SQL Query, here we need to provide the joining condition using the “**on**” operator, and we need to provide the common properties using the “**equals**” operator. Finally, we must project the result to either an anonymous or a named type. In the below code, we are projecting the result to an anonymous type.

```
// Outer Data Source
from emp in Employee.GetAllEmployees()

//Inner Data Source
join address in Address.GetAllAddresses()

//fetching the data using on condition
on emp.AddressId equals address.ID

//Projecting the result set
select new
{
    EmployeeName = emp.Name,
    AddressLine = address.AddressLine
}
```

The complete example code is given below. Once the Join Operation is done, you can access the elements using a for each loop, which is shown in the example below. The following code is self-explained, so please go through the comment lines.

```
using System.Linq;
using System;
namespace LINQJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Inner Join Between Employees and Addresses Collection
            var JoinUsingQS = (from emp in Employee.GetAllEmployees() //Outer
                               join address in Address.GetAllAddresses() //Joining Inner Data Source
                               on emp.AddressId equals address.ID //Joining Condition
                               select new //Projecting the Result to an Anonymous Type
                               {
                                   EmployeeName = emp.Name,
                                   AddressLine = address.AddressLine
                               }).ToList();
            //Accessing the Elements using Foreach Loop
```

```

        foreach (var employee in JoinUsingQS)
        {
            Console.WriteLine($"Name :{employee.EmployeeName}, Address :
{employee.AddressLine}");
        }
        Console.ReadLine();
    }
}

```

Now, run the application, and you will get the following output.

```

Name :Preety, Address : AddressLine1
Name :Priyanka, Address : AddressLine2
Name :Anurag, Address : AddressLine3
Name :Pranaya, Address : AddressLine4
Name :Hina, Address : AddressLine5
Name :Santosh, Address : AddressLine9
Name :Raja, Address : AddressLine10
Name :Sudhanshu, Address : AddressLine11

```

Instead of projecting the Result to an Anonymous Type, we can use the Query Syntax to project the result to a named type. We have already created a type called EmployeeAddress. Let us project the result to the EmployeeAddress type. So, modify the Main method of the Program class as follows.

```

using System.Linq;
using System;
namespace LINQJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Inner Join Between Employees and Addresses Collection
            var JoinUsingQS = (from emp in Employee.GetAllEmployees() //Outer
                               join address in Address.GetAllAddresses() //Join
                               on emp.AddressId equals address.ID //Joining Condition
                               select new EmployeeAddress //Projecting the Result to EmployeeAddress Type
                               {
                                   EmployeeName = emp.Name,
                                   AddressLine = address.AddressLine
                               }).ToList();
            //Accessing the Elements using Foreach Loop
            foreach (var employee in JoinUsingQS)
            {
                Console.WriteLine($"Name :{employee.EmployeeName}, Address :
{employee.AddressLine}");
            }
        }
    }
}

```

```

    }
    Console.ReadLine();
}
}
}

```

Optimizing Performance for Large Datasets using LINQ Inner Join

Optimizing performance for large datasets when using LINQ (Language Integrated Query) in C# can be crucial, especially when performing operations like inner joins that can be computationally expensive. LINQ makes it easy to perform complex queries on collections of objects, but without careful consideration, these queries can lead to performance bottlenecks. Here's an example of how to optimize a LINQ query performing an inner join on large datasets, including some tips to enhance its performance:

Basic LINQ Inner Join Example

First, let's start with a basic example of an inner join using LINQ:

```

using System.Linq;
using System;
namespace LINQJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Inner Join Between Employees and Addresses Collection
            var joinedList = from employee in Employee.GetAllEmployees() //Collection of Employees
                             join address in Address.GetAllAddresses() //Collection of Addresses
                             on employee.AddressId equals address.ID //Joining on AddressId
                             //Projecting the Result to an anonymous type
                             select new { employee.ID, employee.Name, address.ID, address.AddressLine };

            //Accessing the Elements using Foreach Loop
            foreach (var empAddress in joinedList)
            {
                Console.WriteLine($"Id: {empAddress.ID}, Name :{empAddress.Name}, Address : {empAddress.AddressLine}");
            }
            Console.ReadLine();
        }
    }
}

```

The above example joins two collections based on a common key. However, this straightforward approach might not be the most efficient when dealing with large datasets.

Why is the Basic LINQ Inner Join Not Efficient when Datasets are Large?

When dealing with large datasets, the basic LINQ inner join example might not be the most efficient for several reasons related to how LINQ processes the data, the structure of the data collections, and the computational complexity of the operations involved. Here's a breakdown of why it might not be the most efficient approach:

Sequential Processing

The basic LINQ inner join example processes data sequentially. When operating on large datasets, sequential processing can lead to longer execution times because it doesn't take advantage of modern multi-core processors that can handle parallel operations. Processing the data in parallel (using PLINQ, for example) can significantly reduce the time taken by utilizing multiple cores to perform operations concurrently.

Non-Indexed Collections

LINQ's join operation on large datasets can become inefficient if the underlying collections are not indexed. The basic example doesn't specify the data structure of the collections, but if they are `List<T>` or similar non-indexed collections, the operation can result in a full scan of one or both collections for each join operation. This leads to a computational complexity of $O(n*m)$, where n and m are the sizes of the two collections. On the other hand, using indexed collections like `Dictionary<TKey, TValue>` or `Lookup<TKey, TValue>` can significantly improve performance because lookups can be done in $O(1)$ time.

Lack of Optimization Opportunities

The query syntax in LINQ, while readable and convenient, might not always offer the same level of optimization opportunities as method syntax. The compiler translates query syntax into method syntax, and in this translation, certain optimizations that could have been applied might not be evident or utilized. By directly using the method syntax, developers have more explicit control over the operations, potentially leading to more efficient execution plans.

Optimizing the LINQ Query Using Method Syntax

Use the Join Method Directly. The method syntax can sometimes offer more optimization opportunities compared to the query syntax:

```
using System.Linq;
using System;
namespace LINQJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Inner Join Between Employees and Addresses Collection
            var optimizedJoin =
Employee.GetAllEmployees().Join(Address.GetAllAddresses(),
                                employee => employee.AddressId,
                                address => address.ID,
                                (employee, address) => new { employee.ID,
employee.Name, address.AddressLine });
```

```

        //Accessing the Elements using Foreach Loop
        foreach (var empAddress in optimizedJoin)
        {
            Console.WriteLine($"Id: {empAddress.ID}, Name :{empAddress.Name}
: {empAddress.AddressLine}");
        }
        Console.ReadLine();
    }
}

```

Optimized the LINQ Query By Ensuring Collections Are Indexed

If your data resides in a List<T>, every lookup will be O(n). Consider using a Dictionary<TKey, TValue> or Lookup<TKey, TValue> for one or both of the collections you're joining if the key does not already index them. This change can significantly improve performance because lookups in a dictionary or lookup are O(1).

The following example demonstrates a way to efficiently join a collection of employees with their respective addresses using a dictionary for optimization.

```

using System.Linq;
using System;
namespace LINQJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Inner Join Between Employees and Addresses Collection
            var addressDictionary = Address.GetAllAddresses().ToDictionary(addr
address.ID);
            var optimizedJoinWithDictionary = from employee in Employee.GetAll
join address in addressDictionary on
employee.AddressId equals address.Key
select new { employee.ID, employee.N
Address = address.Value };

            //Accessing the Elements using Foreach Loop
            foreach (var empAddress in optimizedJoinWithDictionary)
            {
                Console.WriteLine($"Id: {empAddress.ID}, Name :{empAddress.Name}
{empAddress.Address.AddressLine}");
            }
            Console.ReadLine();
        }
    }
}

```

Note: Here, we use address.Value because addressDictionary is a dictionary. Each item in the join represents a KeyValuePair, where Key is the address ID and Value is the address object itself.

Optimized the LINQ Join By Processing the Operation in Parallel

For extremely large datasets, consider using PLINQ (Parallel LINQ) to utilize multiple cores. Keep in mind that PLINQ can improve performance for CPU-bound operations but might introduce overhead for smaller datasets or operations that are not CPU-intensive.

```
using System.Linq;
using System;
namespace LINQJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Inner Join Between Employees and Addresses Collection

            var parallelJoin = Employee.GetAllEmployees().AsParallel()
                .Join(Address.GetAllAddresses().AsParallel(),
                    employee => employee.AddressId,
                    address => address.ID,
                    (employee, address) => new { employee.ID, employee.Name,
                    address.AddressLine });

            //Accessing the Elements using Foreach Loop
            foreach (var empAddress in parallelJoin)
            {
                Console.WriteLine($"Id: {empAddress.ID}, Name :{empAddress.Name}, Address : {empAddress.AddressLine}");
            }
            Console.ReadLine();
        }
    }
}
```

When working with large datasets, the key to optimizing LINQ queries is to:

- Prefer method syntax for more complex queries.
- Use indexed collections such as dictionaries for faster lookups.
- Consider parallel processing for CPU-bound operations.

When Should We Use LINQ Inner Join in C#?

Using LINQ Inner Join in C# is beneficial when you combine data from two sources (like collections, arrays, or database tables) based on a common key. An Inner Join will only include elements from both sources with matching keys. Here are specific situations when you might use an LINQ Inner Join:

- **Combining Related Data:** When you have two datasets related by a common attribute, you need to combine them to retrieve a set of records that exist in both. For example, when you combine customer and order data, each order is linked to a specific customer by a customer ID.

- **Data Transformation:** When you need to transform data from two sources into a new form where elements from both sources are required. For example, joining a list of products and a list of categories to display products along with their category names.
- **Querying Databases:** LINQ to SQL or Entity Framework queries involve data from multiple tables related through foreign keys. Inner Joins are essential for fetching normalized data spread across different tables.
- **Filtering Data Based on Another Collection:** This is a common requirement in data processing tasks. You want to filter items from one collection based on their presence or matching criteria in another collection.
- **Data Analysis and Reporting:** In scenarios where you need to perform complex data analysis or generate reports that combine data from multiple sources. Inner Join can help consolidate the data for analysis or reporting purposes.
- **Optimizing Performance for Large Datasets:** Sometimes, manually iterating over large datasets to find matching items can be inefficient. Using LINQ Inner Join can optimize these operations, as LINQ queries are often optimized for performance, especially when working with LINQ to Entities or LINQ to SQL.

In the next article, I will discuss [Joining three Data Sources using the LINQ Method and Query Syntax in C#](#) with Examples. In this article, I explain LINQ Inner Join in C# using query and method syntax. I hope you understand how to perform the LINQ Inner Join in C# using Method and Query Syntax with two Data Sources.